FactLens

An advanced RAG-based document analysis assistant focused on explainable, verifiable answers.

Upload one or more documents (.pdf, .docx, .txt, .md), ask a question, and review the answer alongside its verification report. Or load a bundled example and press Submit.

Example

Load example

Documents

Object Slicing.pdf 111.8 KB

Question

Explain object slicing in C++ and its effects

Submit

Object slicing occurs when a derived-class object is copied or assigned **by value** to a base-class object. Only the base portion is copied; members and behavior specific to the derived class are discarded.

For example, passing a `Dog` to `void announce(Animal a)` slices it into an `Animal`. Calling `a.speak()` then uses `Animal::speak()`, not `Dog::speak()`, and the `Dog`'s `barkVolume` is lost.

Effects include:

- **Loss of derived state:** derived-only data members are not preserved.
- **Broken polymorphism:** virtual calls on the sliced base object use the base implementation.
- **Subtle bugs:** the code may compile and run while behaving incorrectly.
- **API and container pitfalls:** functions that take or return base objects by value, or containers such as `std::vector<Animal>`, can silently slice derived objects.

To avoid slicing, use base-class references or pointers for polymorphic objects, such as `std::unique_ptr<Animal>` in a container. For value-based handling of a known set of types, `std::variant` is another option.

Supported: YES **Unsupported Claims:** None **Contradictions:** None **Relevant:** YES **Additional Details:** The answer accurately describes slicing, its effects, the `announce(Animal a)` example, and common ways to avoid it. The context also mentions alternatives such as `clone`, type erasure, and composition, but their omission does not make the answer inaccurate.